



PySpark Series From Basic To Advance

Environment Set Up to Learn PySpark

Databricks Free Edition Set Up

- <https://www.databricks.com/learn/free-edition>

Databricks Community Edition Set Up(Databricks has phased out the older Community Edition, replacing it with a revamped Free Edition.)

- <https://www.databricks.com/try-databricks#account>

Create an Empty RDD

```
emptyRDD=spark.sparkContext.emptyRDD()  
emptyRDD.collect()
```

create an empty rdd using parallelize method

```
rdd2=spark.sparkContext.parallelize([])  
rdd2.collect()
```

Fundamental API's of Spark

- Spark has two fundamental sets of APIs:
 - 1) the low-level “unstructured” APIs -> RDD
 - 2) the higher-level structured APIs. -> Dataset and Dataframe

RDD:-

RDD can be process both structured as well as unstructured data

Dataframe and Dataset:-

Dataframe and Dataset works on structured data.

Note:- PySpark doesn't support Dataset.

History of Spark APIs



RDD Creation

```
#create an rdd using parallelize method with some data inside it
dept=[("Finanne",10),("Marketing",20),("IT",30)]
rdd1=spark.sparkContext.parallelize(dept)
print(rdd1)
rdd1.collect()
```

OR

```
dept=[("Finance",10),("Marketing",20),("IT",30)]
rdd3=sc.parallelize(dept)
```

StructType and StructField

StructType

- StructType is a class that represents the schema of a DataFrame.
- It defines a list of StructField objects, each of which describes a single column in the DataFrame.
- This is used to define the schema explicitly, specifying the name, data type, and other properties of each column.

StructField

- StructField is a class that represents a single field (or column) in a StructType.
- It specifies the name of the field, the data type of the field, and whether the field can contain null values.

pyspark.sql.types

The **pyspark.sql.types** module in PySpark provides a set of classes and functions for defining the schema of DataFrames. Both StructType and StructField are part of this module.

To use StructType, StructField in your code to define dataframe you need to first import it using below code:-

```
from pyspark.sql.types import StructType, StructField
```

Dataframe Creation in PySpark

```
#create a dataframe with rdd

name=[("Arun","Kumar","Shaw"),("Anup","Kumar","Shaw"),("Gautam","Kumar","Sharma")]
rdd4=sc.parallelize(name)

from pyspark.sql.types import *
#from pyspark.sql.types import IntegerType(),StringType(),StructType(),StructField()

schema = StructType([
    StructField('firstname', StringType(), True),
    StructField('middlename', StringType(), True),
    StructField('lastname', StringType(), True)
])

df=spark.createDataFrame(rdd4,schema)
df.show()
```

Different Ways To Select Columns in PySpark

1. Using select()

```
# Sample DataFrame
data = [("Alice", 34), ("Bob", 45)]
df = spark.createDataFrame(data, ["name", "age"])

# Selecting columns
selected_df = df.select("name") # Selects only the 'name' column
selected_df.show()
```

2. Using Column Objects

You can also use col() to refer to columns

```
from pyspark.sql.functions import col

selected_df = df.select(col("name"), col("age"))
selected_df.show()
```


Different Ways To Select Columns in PySpark Cont..

3. Using selectExpr()

If you need to perform some expressions or transformations, you can use selectExpr().

```
selected_df = df.selectExpr("name", "age + 1 as age_plus_one")
selected_df.show()
```

4. Selecting Multiple Columns

You can pass multiple column names to select().

```
selected_df = df.select("name", "age")
selected_df.show()
```

5. Using filter() or where() with select()

You can combine selection with filtering.

```
filtered_df = df.filter(col("age") > 40).select("name")
filtered_df.show()
```

Different Ways To Select Columns in PySpark Cont..

6. Selecting All Columns

If you want to select all columns, you can use `*` in SQL-like syntax with `selectExpr()`.

```
selected_df = df.selectExpr("*")  
selected_df.show()
```

Summary

- Use `select()` for straightforward column selection.
- Use `col()` for referencing columns in a more explicit way.
- Use `selectExpr()` for expressions or transformations.
- Combine `filter()` with `select()` to get filtered results.

- **concat**
- **Purpose:** Joins multiple columns or string literals together without any separator.
- **Usage:** You can pass multiple columns or string literals as arguments.

```
from pyspark.sql.functions import concat, col
```

```
df = spark.createDataFrame([("Alice", "Smith"), ("Bob", "Brown")],  
    ["first_name", "last_name"])
```

```
result = df.select(concat(col("first_name"),  
    col("last_name")).alias("full_name"))
```

```
result.show()
```

```
+-----+-----+  
|first_name|last_name|  
+-----+-----+  
|      Alice|      Smith|  
|       Bob|      Brown|  
+-----+-----+
```

```
▶ result: pyspark.sql.  
+-----+  
| full_name|  
+-----+  
| AliceSmith|  
|  BobBrown|  
+-----+
```

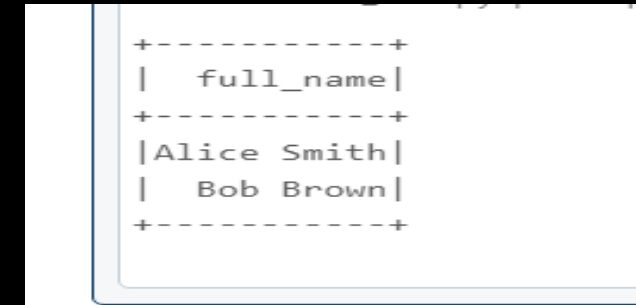
concat_ws

- **Purpose:** Concatenates multiple columns or strings with a specified separator.
- **Usage:** The first argument is the separator, followed by the columns or strings to concatenate.

```
from pyspark.sql.functions import concat_ws
```

```
result_ws = df.select(concat_ws(" ", col("first_name"),  
                               col("last_name")).alias("full_name"))
```

```
result_ws.show()
```



```
+-----+  
| full_name |  
+-----+  
| Alice Smith |  
| Bob Brown |  
+-----+
```

concat vs concat_ws

- **Summary of Differences**

- **Separator:**

- concat has no separator; it directly joins the strings.
- concat_ws requires a separator as the first argument, which it uses between the concatenated strings.

- **Use Case:**

- Use concat when you want a straightforward concatenation.
- Use concat_ws when you need to include a specific separator, such as a space, comma, or any other character.

DataFrame Creation

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
```

```
# Sample data
```

```
data = [  
    (1, "Alice Smith", "alice.smith@example.com", 30),  
    (2, "Bob Johnson", "bob.johnson@example.com", 45),  
    (3, "Charlie Brown", "charlie.brown@example.com", 28),  
    (4, "Daisy Miller", "daisy.miller@example.com", 35),  
    (5, "Eve Davis", "eve.davis@example.com", 22),  
    (4, "Daisy Miller", "daisy.miller@example.com", 35),  
    (5, "Eve Davis", "eve.davis@example.com", 22),  
    (6, "Anup Kumar", "anup.kumar@forumde.com", 28),  
    (7, "Arun Kumar", "arun.kumar@forumde.com", None),  
    (8, "Sandhya", "sandhya@forumde.com", None)  
]
```

DataFrame Creation

Define schema

```
schema = StructType([  
    StructField("customer_id", IntegerType(), True),  
    StructField("name", StringType(), True),  
    StructField("email", StringType(), True),  
    StructField("age", IntegerType(), True)  
])
```

Create DataFrame

```
customers_df = spark.createDataFrame(data, schema)
```

Show the DataFrame

```
customers_df.show()
```

Filter or Where

- To filter rows, use the `filter` or `where` method on a DataFrame to return only certain rows. To identify a column to filter on, use the `col` method or an expression that evaluates to a column.
- `from pyspark.sql.functions import col`
- `df_that_one_customer = df_customer.filter(col("customer_id") == 2)`
- To filter on multiple conditions, use logical operators. For example, `&` and `|` enable you to AND and OR conditions, respectively.
- `df_customer.filter((col("customer_id") == 2) & (col("customer_id") > 3))`
- `df_filtered_customer = df_customer.filter((col("customer_id") == 1) | (col("customer_id") == 3))`

Remove Duplicates

To de-duplicate rows, use `distinct` or `dropDuplicates`, which returns only the unique rows.

```
df_unique = df_customer.distinct().show()
```

Or

```
df_unique = df_customer.dropDuplicates().show()
```

When

```
from pyspark.sql.functions import when
df.withColumn("grade", \
    when((df.salary < 1000), lit("A")) \
        .when((df.salary >= 1000) & (df.salary <= 3000), lit("B")) \
        .otherwise(lit("C")) \
    ).show()
```

PySpark Series Continued...

Window Frame Specification

You can define a window frame using the following functions:

- **rowsBetween(start, end)** — Defines the number of rows to look back (e.g., previous row, next row).
- **rangeBetween(start, end)** — Defines the range of values in a column (useful for range-based windows).

rowsBetween(start, end)

In PySpark, `rowsBetween(start, end)` is used with **window functions** to define a **window frame**. It specifies a range of rows relative to the **current row** for functions like `sum`, `avg`, `min`, `max`, etc.

Syntax

```
windowSpec = Window.partitionBy(...).orderBy(...).rowsBetween(start, end)
```

- `start`: Offset from the current row where the frame begins
- `end`: Offset from the current row where the frame ends

Common Values

- `Window.unboundedPreceding`: All rows before (start of partition)
- `Window.unboundedFollowing`: All rows after (end of partition)
- `Window.currentRow`: Only current row
- `-n`: n rows before the current row
- `+n`: n rows after the current row

Frame	Description
<code>.rowsBetween(-1, 1)</code>	Current row $\pm$ 1
<code>.rowsBetween(Window.unboundedPreceding, Window.currentRow)</code>	From first row to current row
<code>.rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)</code>	Entire partition

Create Dataframe

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col
```

Sample data

```
data = [
    ("A", 1),
    ("A", 2),
    ("A", 3),
    ("A", 4),
    ("B", 1),
    ("B", 2),
    ("B", 3)
]

df = spark.createDataFrame(data, ["group", "value"])
```

```
+-----+-----+
|group|value|
+-----+-----+
|  A  |  1  |
|  A  |  2  |
|  A  |  3  |
|  A  |  4  |
|  B  |  1  |
|  B  |  2  |
|  B  |  3  |
+-----+-----+
```

.rowsBetween(-1, 1)

```
# Define a window partitioned by group and ordered by value
windowSpec = Window.partitionBy("group").orderBy("value").
rowsBetween(-1, 1)
```

```
# Add rolling sum over the window frame
```

```
df_with_sum = df.withColumn("rolling_sum",sum("value").over
(windowSpec))
```

```
df_with_sum.show()
```

```
+-----+-----+-----+
|group|value|rolling_sum|
+-----+-----+-----+
|  A|  1|      3| # 1 (current) + 2 (next)
|  A|  2|      6| # 1 + 2 + 3
|  A|  3|      9| # 2 + 3 + 4
|  A|  4|      7| # 3 + 4
|  B|  1|      3| # 1 + 2
|  B|  2|      6| # 1 + 2 + 3
|  B|  3|      5| # 2 + 3
+-----+-----+-----+
```

rowsBetween(Window.unboundedPreceding,Window.currentRow)

```
# Define window: partition by group, order by value, cumulative till
current row
from pyspark.sql.window import Window

windowSpec = Window.partitionBy("group").orderBy("value") \
.rowsBetween(Window.unboundedPreceding, Window.currentRow)

# Add cumulative sum column df_with_cumsum =
df.withColumn("cumulative_sum", sum("value").over(windowSpec))
df_with_cumsum.show()
```

+---+---+---+---+			
group	value	cumulative_sum	
+---+---+---+---+			
A	1	1	
A	2	3	# 1 + 2
A	3	6	# 1 + 2 + 3
A	4	10	# 1 + 2 + 3 + 4
B	1	1	
B	2	3	# 1 + 2
B	3	6	# 1 + 2 + 3
+---+---+---+---+			

Use Case:

This is a **cumulative sum**, very common in financial or trend analysis (e.g. running total sales, scores, balances).

rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)

Define window: partition by group, order by value, cumulative till current row

```
from pyspark.sql.window import Window
```

```
windowSpec = Window.partitionBy("group").orderBy("value") \
```

```
    .rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)
```

Add cumulative sum column

```
df_with_cumsum = df.withColumn("cumulative_sum", sum("value").over(windowSpec))
```

```
df_with_cumsum.show()
```

```
+-----+-----+-----+
|group|value|cumulative_sum|
+-----+-----+-----+
|  A  |  1  |          10 | # 1 + 2 + 3 + 4
|  A  |  2  |          10 |
|  A  |  3  |          10 |
|  A  |  4  |          10 |
|  B  |  1  |           6 | # 1 + 2 + 3
|  B  |  2  |           6 |
|  B  |  3  |           6 |
+-----+-----+-----+
```

rangeBetween(start, end)

In PySpark, `rangeBetween(start, end)` is used within a **Window specification** to define a **range-based window frame** for performing operations like `sum()`, `avg()`, etc., **based on values of a column** (usually ordered numerically or by date).

Important Points

=====

- start and end define the **range of values** relative to the current row's **orderBy column value**.
- `rangeBetween` works **only with numeric or timestamp types**.
- Unlike `rowsBetween`, which looks at physical row positions, `rangeBetween` considers **column value ranges**.

Syntax

=====

```
from pyspark.sql.window import Window
```

```
window_spec = Window.partitionBy("group_column").orderBy("order_column") \
    .rangeBetween(start, end)
```

rangeBetween with numeric type

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.partitionBy("group").orderBy("value") \
    .rangeBetween(-1, 1)

df.withColumn("range_sum", sum("value").over(window_spec)).show()
```

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum
```

```
data = [("A", 1), ("A", 2), ("A", 3), ("A", 4), ("B", 1), ("B", 3), ("B", 5)]
df = spark.createDataFrame(data, ["group", "value"])
```

group	value
A	1
A	2
A	3
A	4
B	1
B	3
B	5

```
window_spec = Window.partitionBy("group").orderBy("value").rangeBetween(-1, 1)
df.withColumn("range_sum", sum("value").over(window_spec)).show()
```

group	value	range_sum
A	1	3
A	2	6
A	3	9
A	4	7
B	1	1
B	3	3
B	5	5

rangeBetween with timestamp type



ForumDE
Code Your Career

```
from pyspark.sql.functions import col, sum, to_timestamp, unix_timestamp
```

```
# Sample data
```

```
data = [  
    ("A", "2024-06-01 10:00:00", 10),  
    ("A", "2024-06-02 10:00:00", 20),  
    ("A", "2024-06-03 10:00:00", 30),  
    ("A", "2024-06-04 10:00:00", 40),  
    ("B", "2024-06-02 10:00:00", 20),  
    ("B", "2024-06-03 10:00:00", 30),  
    ("B", "2024-06-04 10:00:00", 40)
```

group	ts	value	ts_long
A	2024-06-01 10:00:00	10	1717236000000
A	2024-06-02 10:00:00	20	1717322400000
A	2024-06-03 10:00:00	30	1717408800000
A	2024-06-04 10:00:00	40	1717495200000
B	2024-06-02 10:00:00	20	1717322400000
B	2024-06-03 10:00:00	30	1717408800000
B	2024-06-04 10:00:00	40	1717495200000

```
df = spark.createDataFrame(data, ["group", "ts", "value"]) \  
.withColumn("ts", to_timestamp("ts")) \  
.withColumn("ts_long", unix_timestamp("ts") * 1000) # convert to milliseconds  
df.show()
```

```
# One day in milliseconds
```

```
ONE_DAY_MS = 86400000
```

```
from pyspark.sql.window import Window
```

```
window_spec = Window.partitionBy("group") \
    .orderBy("ts_long") \
    .rangeBetween(-ONE_DAY_MS, ONE_DAY_MS)
```

```
df.withColumn("range_sum", sum("value").over(window_spec)).select("group", "ts", "value",
    "range_sum").show()
```

group	ts	value	range_sum
A	2024-06-01 10:00:00	10	30
A	2024-06-02 10:00:00	20	60
A	2024-06-03 10:00:00	30	90
A	2024-06-04 10:00:00	40	70
B	2024-06-02 10:00:00	20	50
B	2024-06-03 10:00:00	30	90
B	2024-06-04 10:00:00	40	70

Summary

Function	Works on	Use values or row index?
rangeBetween	Ordered values	Uses actual values
rowsBetween	Row positions	Uses row index

Window Functions - First and Last

```
from pyspark.sql.functions import first, last
```

```
from pyspark.sql.window import Window
```

```
# Sample Data
```

```
data = [
```

```
    ("A", "2024-01-01", 100),
```

```
    ("A", "2024-01-02", 200),
```

```
    ("A", "2024-01-03", 300),
```

```
    ("B", "2024-01-01", 150),
```

```
    ("B", "2024-01-02", 250),
```

```
]
```

```
df = spark.createDataFrame(data, ["group", "date", "value"])
```

```
+-----+-----+-----+
|group|      date|value|
+-----+-----+-----+
|    A|2024-01-01|  100|
|    A|2024-01-02|  200|
|    A|2024-01-03|  300|
|    B|2024-01-01|  150|
|    B|2024-01-02|  250|
+-----+-----+-----+
```



```
window_spec = Window.partitionBy("group").orderBy("date")
from pyspark.sql.functions import first, last
df_with_first_last = df.withColumn("first_value", first("value").over(window_spec)) \
.withColumn("last_value", last("value").over(window_spec))
df_with_first_last.show()
```

Note:-

In the **output of the code** , the last() function **returns the last value up to the current row** due to the **default window frame**, not the last value of the *entire partition*.

group	date	value	first_value	last_value
A	2024-01-01	100	100	100
A	2024-01-02	200	100	200
A	2024-01-03	300	100	300
B	2024-01-01	150	150	150
B	2024-01-02	250	150	250

Important:-

window_spec = Window.partitionBy("group").orderBy("date") has frame of:-

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

So last("value") gives the last value **up to the current row, not the last value of the partition**.

We need to **change the frame** using `rowsBetween` to get the entire partition in view.

```
# Define window with full partition
```

```
window_spec = Window.partitionBy("group").orderBy("date") \  
.rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)
```

```
# Apply first() and last() over full partition
```

```
from pyspark.sql.functions import first, last
```

```
df_with_first_last = df.withColumn("first_value", first("value").over(window_spec)) \  
.withColumn("last_value", last("value").over(window_spec))
```

```
df_with_first_last.show()
```

group	date	value	first_value	last_value
A	2024-01-01	100	100	300
A	2024-01-02	200	100	300
A	2024-01-03	300	100	300
B	2024-01-01	150	150	250
B	2024-01-02	250	150	250

Window Functions - sum(), avg(), max(), and min()

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, avg, max, min, to_date
```

```
# Create DataFrame
```

```
data = [
    ("A", "2024-01-01", 100),
    ("A", "2024-01-02", 200),
    ("A", "2024-01-03", 300),
    ("B", "2024-01-01", 150),
    ("B", "2024-01-02", 250),
]
```

```
+-----+-----+-----+
|group|      date|value|
+-----+-----+-----+
|  A|2024-01-01|  100|
|  A|2024-01-02|  200|
|  A|2024-01-03|  300|
|  B|2024-01-01|  150|
|  B|2024-01-02|  250|
+-----+-----+-----+
```

```
df = spark.createDataFrame(data, ["group", "date", "value"])
df = df.withColumn("date", to_date("date"))
```

```
# Define window
```

```
window_spec = Window.partitionBy("group").orderBy("date") \  
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
```

```
# Apply window functions
```

```
from pyspark.sql.functions import sum, avg, max, min
```

```
df_with_aggregates = df.withColumn("running_sum",  
    sum("value").over(window_spec)) \  
    .withColumn("running_avg", avg("value").over(window_spec)) \  
    .withColumn("running_max", max("value").over(window_spec)) \  
    .withColumn("running_min", min("value").over(window_spec))
```

```
df_with_aggregates.show()
```

group	date	value	running_sum	running_avg	running_max	running_min
A	2024-01-01	100	100	100.0	100	100
A	2024-01-02	200	300	150.0	200	100
A	2024-01-03	300	600	200.0	300	100
B	2024-01-01	150	150	150.0	150	150
B	2024-01-02	250	400	200.0	250	150

Window Functions - ntile, cume_dist and percent_rank

```
from pyspark.sql.functions import ntile, cume_dist, percent_rank
from pyspark.sql.window import Window
```

```
# Sample DataFrame
```

```
data = [ ("Alice", 95), ("Bob", 85), ("Cathy", 80), ("David", 70), ("Emma", 60) ]
```

```
df = spark.createDataFrame(data, ["name", "score"])
```

```
# Define window specification (sorted by score)
```

```
window_spec = Window.orderBy("score")
```

```
df_with_metrics = df.withColumn("tile_3", ntile(3).over(window_spec)) \
.withColumn("cume_dist", cume_dist().over(window_spec)) \
.withColumn("percent_rank", percent_rank().over(window_spec))
```

```
result df_with_metrics.show()
```

name	score
Emma	60
David	70
Cathy	80
Bob	85
Alice	95

name	score	tile_3	cume_dist	percent_rank
Emma	60	1	0.2	0.0
David	70	1	0.4	0.25
Cathy	80	2	0.6	0.5
Bob	85	2	0.8	0.75
Alice	95	3	1.0	1.0

name	score	tile_3	cume_dist	percent_rank
Emma	60	1	0.2	0.0
David	70	1	0.4	0.25
Cathy	80	2	0.6	0.5
Bob	85	2	0.8	0.75
Alice	95	3	1.0	1.0

Suppose I am Cathy !

$$\text{CUME_DIST} = \frac{\text{Number of Rows With Value} \leq \text{Current Row}}{\text{Total Number of Rows}}$$

$$\text{PERCENT_RANK} = \frac{\text{Rank} - 1}{\text{Total Rows} - 1}$$

Scenario

"How many people are below me in performance?"

"Where do I stand on a 0–1 scale rank-wise?"

Use

cume_dist

percent_rank

Real-Life Analogy:

- cume_dist() is like saying: **"I'm in the top 60% of the class."**
- percent_rank() is like saying: **"I ranked better than 50% of the class."**

Difference Between GroupBy And PartitionBy

Input DataFrame

Region	Sales
East	1000
East	2000
East	2000
West	3000
West	3000
West	3000

GroupBy Output

Region	Total_Sales
East	5000
West	9000

```
grouped_df =
```

```
df.groupBy("Region").agg(F.sum("Sales").alias("Total_
Sales")) grouped_df.show()
```

- Rows are grouped and aggregated.
- Output is summarized per group.
- The number of rows is reduced.

PartitionBy Output

Region	Sales	Rank
East	2000	1
East	2000	1
East	1000	3
West	3000	1
West	3000	1
West	3000	1

```
# Add rank within each region
```

```
ranked_df = df.withColumn("Rank",
F.rank().over(windowSpec)) ranked_df.show()
```

- All original rows are retained.
- Data is logically partitioned in memory, not reduced.



ForumDE
Code Your Career

New Azure batch starting on 28st, June, 2025



Microsoft
Azure



SQL



Azure Data
Factory



databricks



NEW BATCH

ANNOUNCEMENT

Updates of Our New

AZURE DATA ENGINEERING

COURSE Duration - 4 Months

Job Oriented IT Training



Mr. Arun Kumar



+91-70429 28331, +91-98018 30173

Website:- <https://forumde.in/>



<https://forumde.in>

Different Kinds Of Joins In PySpark

Join Type	Returned Rows	Returned Columns
Inner	Matching rows	Both DataFrames
Left Outer	All left + matching right	Both DataFrames
Right Outer	All right + matching left	Both DataFrames
Full Outer	All rows from both sides	Both DataFrames
Left Semi	Matching rows from left	Left DataFrame only
Right Semi	Matching rows from right	Right DataFrame only
Left Anti	Non-matching rows from left	Left DataFrame only
Right Anti	Non-matching rows from right	Right DataFrame only

Syntax Of All Kinds Of Join

```
df1.join(df2, on="id", how="inner")
```

```
df1.join(df2, on="id", how="left")
```

```
df1.join(df2, on="id", how="right")
```

```
df1.join(df2, on="id", how="outer")
```

```
df1.join(df2, on="id", how="left_semi")
```

```
df1.join(df2, on="id", how="left_anti")
```

```
df2.join(df1, on="id", how="right_semi")
```

```
df2.join(df1, on="id", how="right_anti")
```

```
df1.crossJoin(df2)
```

Input Dataframe

Left DataFrame (df1)

id	name
1	Alice
2	Bob
3	Charlie
4	David

Right DataFrame (df2)

id	city
1	New York
2	London
5	Paris

Inner Join

id	name	city
1	Alice	New York
2	Bob	London

Left Join

id	name	city
1	Alice	New York
2	Bob	London
3	Charlie	null
4	David	null

Input Dataframe

Left DataFrame (df1)

id	name
1	Alice
2	Bob
3	Charlie
4	David

Right DataFrame (df2)

id	city
1	New York
2	London
5	Paris

Right Join

id	name	city
1	Alice	New York
2	Bob	London
5	null	Paris

All rows from df2; unmatched df1 values are null.

Full Outer Join

id	name	city
1	Alice	New York
2	Bob	London
3	Charlie	null
4	David	null
5	null	Paris

All rows from both, with nulls where no match.

Input Dataframe

Left DataFrame (df1)

id	name
1	Alice
2	Bob
3	Charlie
4	David

Right DataFrame (df2)

id	city
1	New York
2	London
5	Paris

Input Dataframe

Left Semi Join

id	name
1	Alice
2	Bob

Only df1 rows that have a match in df2. No df2 columns.

Left Anti Join

id	name
3	Charlie
4	David

Only df1 rows that do NOT have a match in df2.

Left DataFrame (df1)

id	name
1	Alice
2	Bob
3	Charlie
4	David

Right DataFrame (df2)

id	city
1	New York
2	London
5	Paris

Input Dataframe

Left DataFrame (df1)

id	name
1	Alice
2	Bob
3	Charlie
4	David

Right DataFrame (df2)

id	city
1	New York
2	London
5	Paris

Right Semi Join

id	city
1	New York
2	London

Right Semi Join returns only rows from df2 that have a match in df1. It acts like a filter on df2.

Right Anti Join

id	city
5	Paris

Right Anti Join returns only rows from df2 that have NO match in df1.

Input Dataframe

Cross Join

id_df1	name	id_df2	city
1	Alice	1	New York
1	Alice	2	London
1	Alice	5	Paris
2	Bob	1	New York
2	Bob	2	London
2	Bob	5	Paris
...	...	...	...

Left DataFrame (df1)

id	name
1	Alice
2	Bob
3	Charlie
4	David

Right DataFrame (df2)

id	city
1	New York
2	London
5	Paris

Union and Union All In PySpark

- union() behaves like SQL's UNION ALL.
- PySpark does NOT have a separate unionAll() method

Code For Union:-

```
df_union = df1.union(df2)
```

What About SQL-Style UNION (without duplicates)?

In SQL, Union removes the duplicates but In PySpark Union works like UnionAll in SQL.

To remove duplicates in PySpark, we need to use below code:-

```
df_union_distinct = df1.union(df2).distinct()
```


Summary

SQL Equivalent	PySpark Method	Removes Duplicates?
<code>UNION ALL</code>	<code>df1.union(df2)</code>	✗ No
<code>UNION (distinct)</code>	<code>df1.union(df2).distinct()</code>	✓ Yes

Requirements for union() in PySpark:

- 1) Schemas must match exactly (same number of columns and types).
- 2) If column names differ but structure is the same, align them.

Different Ways To Create RDD In PySpark

Method1- From an existing collection

```
# From an existing collection
data = [1, 2, 3, 4, 5]
rdd1 = spark.sparkContext.parallelize(data)
rdd1.collect()
```

Method2- From a dataframe

```
data = [
    (1, "Alice", 29),
    (2, "Bob", 35),
    (3, "Cathy", 23)
]
columns = ["id", "name", "age"]
df = spark.createDataFrame(data, columns)
display(df)
rdd2=df.rdd
rdd2.collect()
```

Method 3-From another RDD

```
data = [1, 2, 3, 4, 5]
rdd1 = spark.sparkContext.parallelize(data)
# From another RDD (transformation)
rdd3 = rdd1.map(lambda x: x * 2)
rdd3.collect()
```

Method 4-Reading From External Sources Using textFile

```
# From an external file
rdd2 = spark.sparkContext.textFile('dbfs:/FileStore/tables/data_engineering.txt')
rdd2.collect()
```

Method 5- Using wholeTextFiles

```
# Using wholeTextFiles (for reading files as (filename, content) pairs)
rdd4 = spark.sparkContext.wholeTextFiles('dbfs:/FileStore/tables')
rdd4.collect()
```

What is Spark?

1) Compute Engine: A Compute Engine is a system that reads data, processes it and produces results — often in a distributed and scalable way.

2) Unified: support a wide range of data analytics tasks, ranging from simple data loading and SQL queries to machine learning and streaming computation, over the same computing engine.

3) Speed: Spark provides in memory storage for intermediate computations and limited disk I/O, this gives it a huge performance boost. In Spark processing happens in RAM.

Run workloads 100x faster.

GraphX Representation

```
Alice — Bob — Charlie
 \           /
  Dave — Eve
```

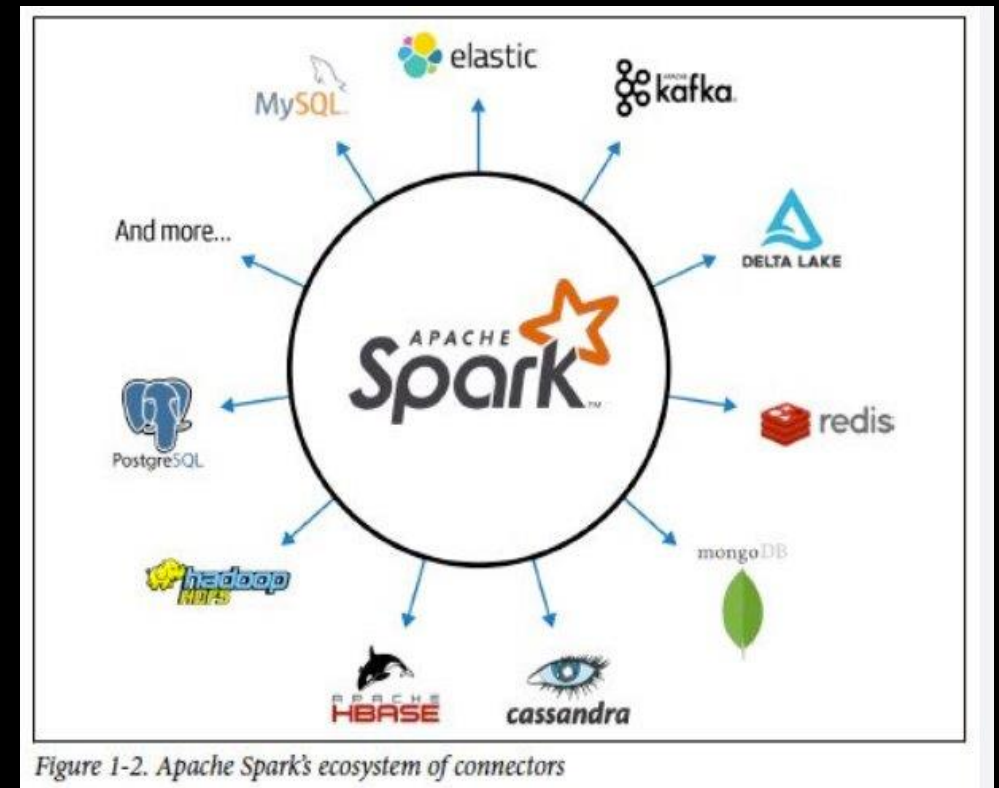
Friend of Friend suggestion system of facebook.



4)Ease of Use: (Write applications quickly in Java, Scala, Python, R, and SQL.

5)Generality: Spark powers a stack of libraries including SQL and DataFrames, MLib for machine learning, GraphX(Graph Analytics) and Spark Streaming You can combine these libraries seamlessly in the same application.

6)Extensibility: Decouples Compute and Storage. It focuses on fast and parallel computation engine than on storage. Storage can be extended to read data from other sources.



7)Can Run Any Cluster Manager:

- YARN
- Mesos
- Standalone
- Kubernetes

8)Open Source

9)Scalable

10)Distributed

11)Batch Data and Streaming Data Processing

Spark's Language APIs

- Scala

Spark is primarily written in **Scala**, making it Spark's “default” language.

- Java

Spark code in Java.

- Python

Spark code can also be written in Python.

- SQL

Spark supports a subset of the **ANSI SQL 2003 standard**.

- R

Spark has two commonly used R libraries: one as a part of Spark core (**SparkR**) and another as an R community-driven package (**sparklyr**).

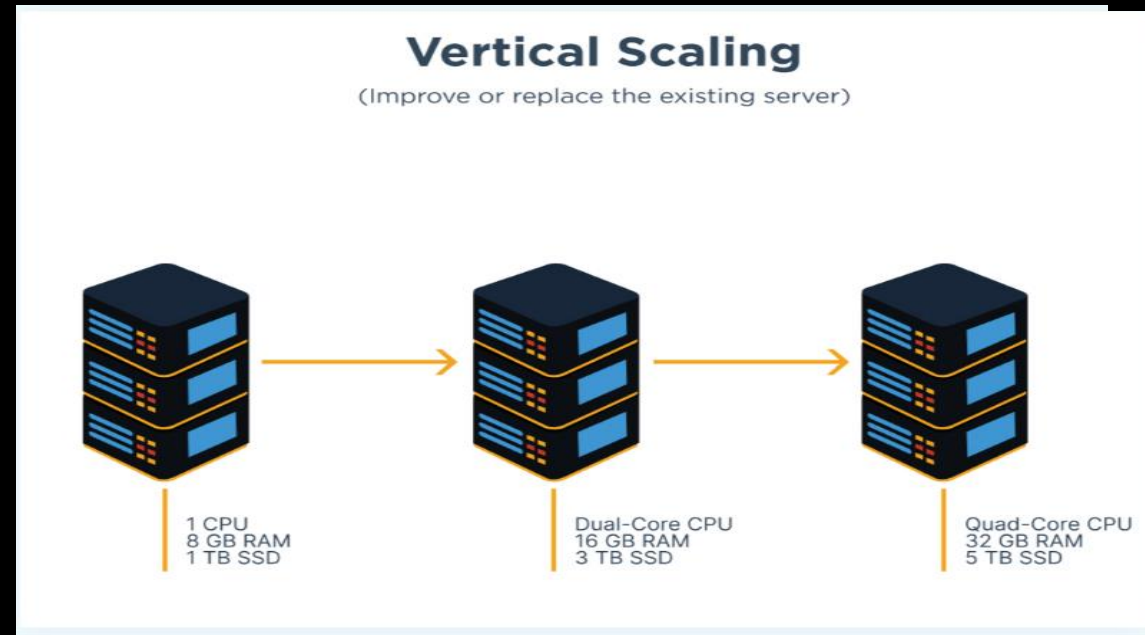
In Python → **PySpark**

In R → **SparkR** or **sparklyr**

In SQL → **Spark SQL**

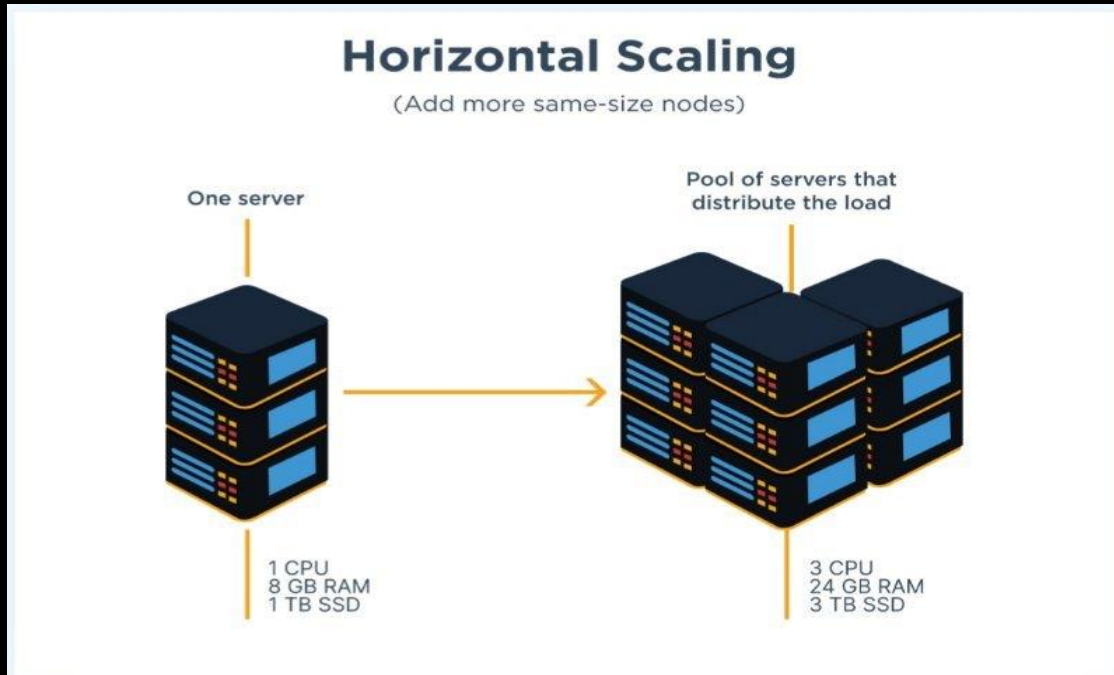
In Java/Scala → **Native Spark APIs**

Vertical Scaling



- **Vertical scaling** (aka **scaling up**) describes adding additional resources to a system so that it meets demand. How is this different from horizontal scaling?
- While horizontal scaling refers to adding additional nodes, **vertical scaling describes adding more power to your current machines.**

Horizontal Scaling



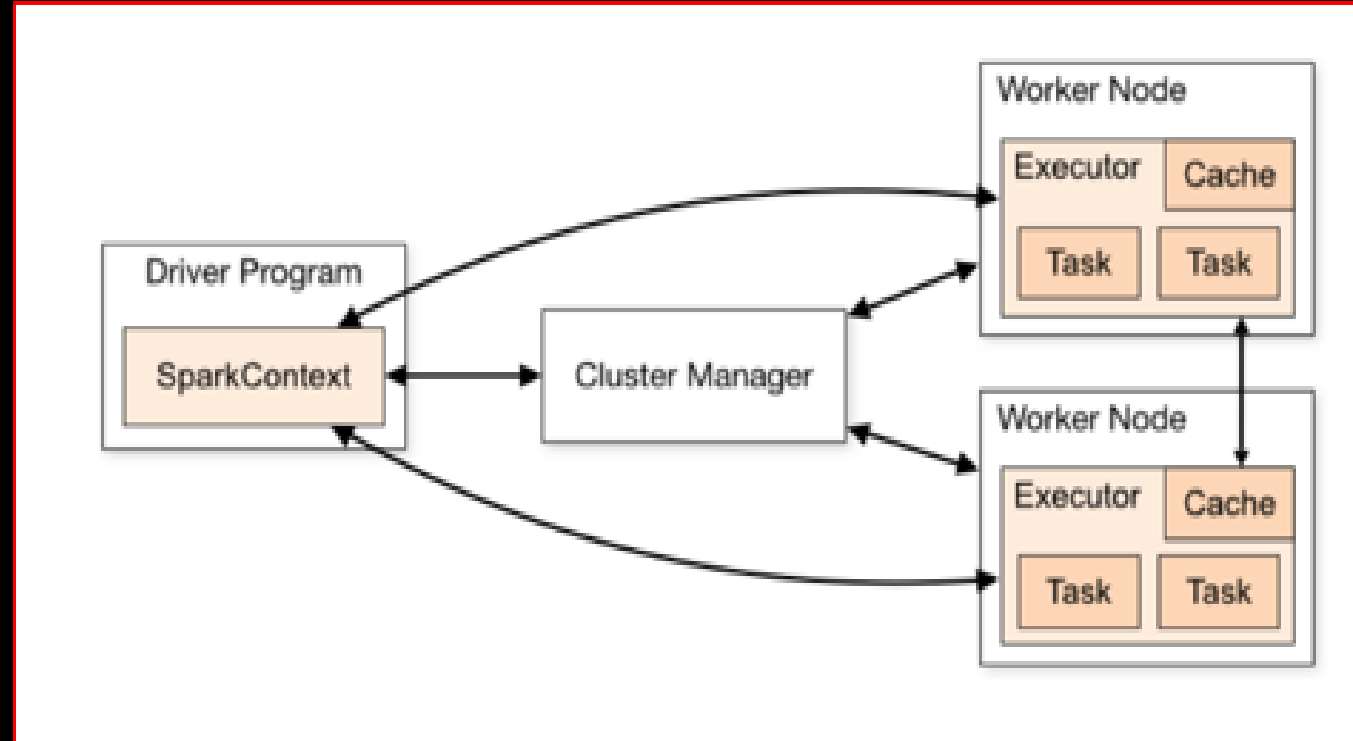
- **Horizontal scaling (aka scaling out)** refers to adding additional nodes or machines to our infrastructure to cope with new demands.
- If you are hosting an application on a server and find that it no longer has the capacity or capabilities to handle traffic, adding a server may be your solution.

Spark Architecture – In Depth

- Spark Applications consist of:-
 1. a driver process
 2. a set of executor processes.

=>Master Slave Architecture

- Driver Node is the **Master**.
- Worker Nodes are **Slave**.



Key Terms

Driver Program- The **Driver Program** is the **main entry point** of your Spark application. It contains the **code you write** to define how your data should be processed.

Cache- In **Spark**, a **cache** is essentially a type of **temporary memory** used to **store intermediate data** in **executor memory**, so it can be **quickly reused** without recomputing or reloading it from disk or the original data source.

Executor- An **executor** is a worker process and distributed process responsible for executing code (tasks) and storing data (caching/shuffling) in a Spark application.

Task- A **task** is a single unit of execution that operates on one partition of data.

Key Terms

SparkContext- The **SparkContext** is the **entry point to the Spark Core engine** — it is **your program's connection to the Spark cluster**. It lets your driver program talk to executors, submit tasks, and access resources.

Cluster Manager- A **Cluster Manager** in Apache Spark is the component responsible for **managing resources (CPU, memory)** across a cluster of machines. It **allocates resources to Spark applications** and coordinates the **execution of jobs** across worker nodes.

1.Driver Process:-

- The driver process runs your main function, sits on a node in the cluster, and is responsible for three things:
 1. maintaining information about the Spark Application
 2. responding to a user's program or input
 3. and analyzing, distributing, and scheduling work across the executors.
- The driver process is the **heart of a Spark Application** and maintains all relevant information during the lifetime of the application.

2.Executor Process:-

- The executors are responsible for actually carrying out the work that the driver assigns them. This means that each executor is responsible for two things:
 1. executing code assigned to it by the driver
 2. reporting the **state of the computation** on that executor back to the driver node

Driver Node

The Driver Node is the machine (or container) where the Driver Program runs.

Responsibilities:

- Hosts the Spark Driver process
- Runs your main Spark application code
- Creates the SparkContext or SparkSession
- Plans jobs, builds DAGs (Directed Acyclic Graphs), breaks them into stages
- Coordinates tasks by sending them to executors on worker nodes
- Collects results from executors

Think of it as:

The **brain** of the Spark application.

Example of a Driver Program (PySpark)

```
from pyspark.sql import SparkSession
```

```
# 1. Start Driver Program (SparkSession)
```

```
spark = SparkSession.builder.appName("SimpleApp").getOrCreate()
```

```
# 2. Define transformations (lazy)
```

```
df = spark.read.csv("/mnt/data/sales.csv", header=True, inferSchema=True)
```

```
filtered_df = df.filter(df["region"] == "South")
```

```
# 3. Trigger an action (starts job)
```

```
filtered_df.show()
```

```
# 4. Stop the Spark session (ends Driver Program)
```

```
spark.stop()
```

This entire script runs on the driver node and is called the driver program.

Key Responsibilities of the Driver Program:

Key Responsibilities of the Driver Program:

1) Starts the Spark Application

- Initializes SparkSession or SparkContext

2) Defines RDD/DataFrame Transformations

- Example: `.map()`, `.filter()`, `.groupBy()`

3) Calls Actions

- Example: `.show()`, `.collect()`, `.count()`
- These trigger jobs in Spark

4) Builds DAG (Directed Acyclic Graph)

- Represents the logical flow of computation

5) Coordinates with the Cluster Manager

- Requests executors and resources

6) Schedules Tasks

- Sends tasks to executors on worker nodes
- Monitors progress and retries if needed

Feature	Driver Program
Runs on	Driver Node
Triggers	Jobs, stages, tasks
Contains	Main logic of your Spark app
Language	Python (PySpark), Scala, Java, R
Failure Impact	If the driver crashes, the entire Spark job fails

Worker Node

A Worker Node is a machine in the cluster that executes tasks and stores data.

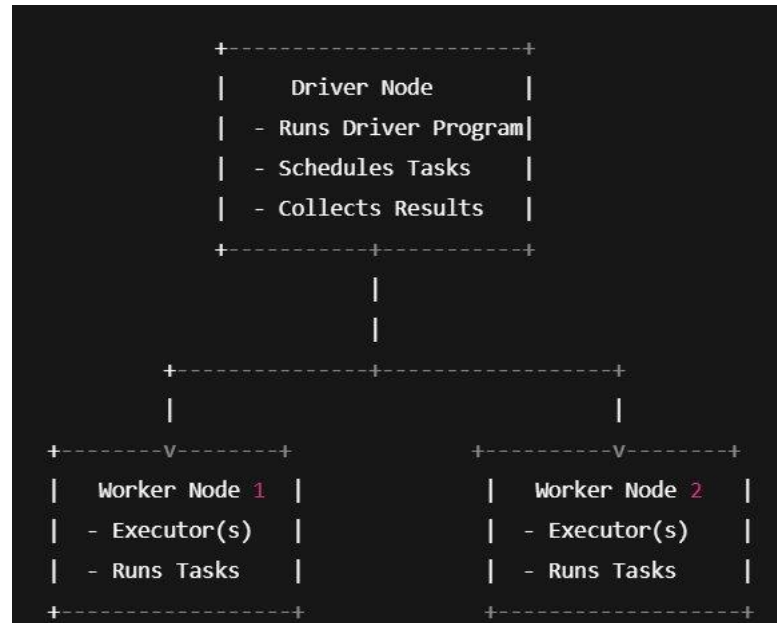
Responsibilities:

- Runs executor processes
- Executors do the actual data processing (map, filter, groupBy, etc.)
- Store data in memory or disk (for caching, shuffling)
- Report task status back to the driver

Think of it as:

The **muscles** of the Spark application.

Summary: Driver Node and Worker Node



Feature	Driver Node	Worker Node
Runs	Driver Program	Executors (one or more)
Role	Coordinates the job	Executes the job
Stores Data	No	Yes (temporary/cache/shuffle data)
Task Execution	No (only schedules)	Yes (per partition)

Feature	Driver Program
Runs on	Driver Node
Triggers	Jobs, stages, tasks
Contains	Main logic of your Spark app
Language	Python (PySpark), Scala, Java, R
Failure Impact	If the driver crashes, the entire Spark job fails

Term	Full Form	Description
SSD	Solid State Drive	Faster, uses flash memory (no moving parts)
HDD	Hard Disk Drive	Slower, uses spinning magnetic disks (mechanical)

Summary: Spark processing happens **primarily in RAM** (in-memory processing).

Summary: Hadoop processing happens **on Disk**, making it slower due to frequent I/O operations.

In Spark, processing happens in RAM (Memory).

In Hadoop, processing happens on disk (HDD/SSD).

How many executors are there in each worker node?

Example Scenario:

Imagine a worker node has:

16 CPU cores

64 GB RAM

And your configuration is:

`spark.executor.cores = 4`

`spark.executor.memory = 16g`

Then each executor will consume:

4 cores

16 GB RAM

So you can fit:

4 executors per worker node

(since $16/4 = 4$ cores per executor and $64/16 = 4$ executors based on memory)

Website: <http://databricks.com/>

What number of tasks that run on each executor ?

In Apache Spark, the **number of tasks that run on each executor** depends on several factors, primarily:

1. Number of Cores per Executor

Each task runs on one core of an executor at a time.

So, number of concurrent tasks = number of cores in that executor.

For example:

If an executor has 4 cores, it can run 4 tasks concurrently.

If you assign 5 tasks, 4 will run in parallel, and 1 will wait until a core is free.

2. Total Number of Tasks = Number of Partitions

The total number of tasks Spark creates = number of partitions of the RDD/DataFrame.

So:

If you have 100 partitions, Spark will create 100 tasks.

These tasks are distributed across all executors, depending on available cores.

Example Scenario:

Cluster: 5 Executors

Each Executor: 4 Cores

Total Cores: $5 \times 4 = 20$

Number of Tasks: 100

Then:

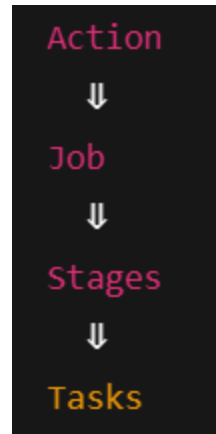
20 tasks will run in parallel (across all executors).

Once a task completes, the executor picks the next one.

The tasks keep getting scheduled until all 100 are completed.

Spark Execution Hierarchy

Job, Stages and Tasks



Component	What It Is	Triggered By	Parallelism
Job	Full action execution plan	Actions like <code>.collect()</code>	1 per action
Stage	Logical step in job	Shuffle boundaries	1+ per job
Task	Unit of execution	One per data partition	Many per stage

Spark Architecture –In Depth

Driver Node(Master)

Worker Node(Slave)

Cluster Manager

Spark Context

Spark Session

Executor

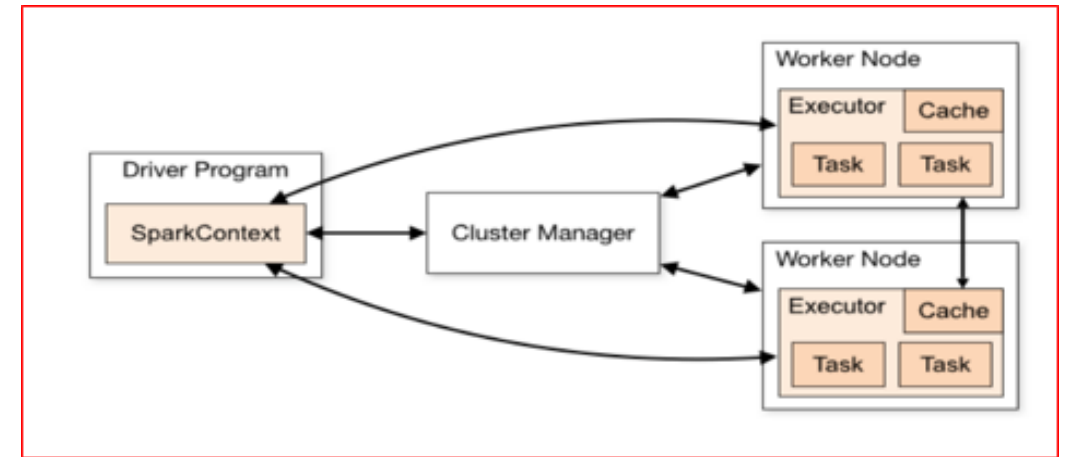
Driver Program

Cache

Component	Description
Worker Node	A physical/logical machine in the cluster
Executor	A JVM process running on a worker node, created by Spark for each app
One worker	Can host multiple executors (if enough resources are available)

In **Apache Spark**, an **executor** is a **distributed process** launched for a Spark application that **runs tasks and stores data** for that application.

An **executor** is a **worker process and distributed process** responsible for **executing code (tasks)** and **storing data (caching/shuffling)** in a Spark application.



=>Master Slave Architecture

- Driver node is the **Master**.
- Worker Nodes are **Slave**.

Key Responsibilities of SparkContext:

The SparkContext is the entry point to the Spark Core engine — it is your program's connection to the Spark cluster. It lets your driver program talk to executors, submit tasks, and access resources.

```
from pyspark import SparkContext
sc = SparkContext("local", "MyApp")
rdd = sc.parallelize([1, 2, 3, 4])
print(rdd.map(lambda x: x * 2).collect())
```

Here, sc is the **SparkContext** that initializes a Spark application named "MyApp".

Task	Description
Cluster Connection	Connects the driver to the Spark cluster manager (Standalone, YARN, Kubernetes)
Resource Allocation	Requests resources (executors) for running tasks
Job & Stage Scheduling	Breaks jobs into stages and tasks
RDD Management	Creates and manages RDDs
Logging & Monitoring	Tracks the status of jobs, stages, and tasks

In modern PySpark (2.x+), you don't usually create `SparkContext` manually — it comes bundled with `SparkSession`.

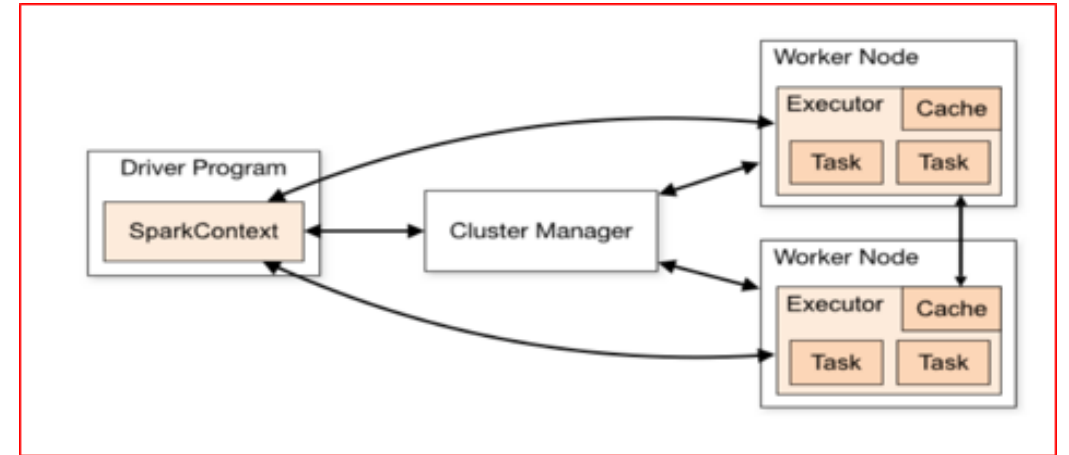
```
spark = SparkSession.builder.appName("MyApp").getOrCreate()  
sc = spark.sparkContext # Access SparkContext from SparkSession
```

Term	Description
SparkContext	Connects your app to Spark cluster; manages jobs, tasks, and RDDs
Runs On	Driver Node
Creates	RDDs, accumulators, broadcast variables
Still Used?	Yes, but mostly behind the scenes via SparkSession

Spark Context vs Spark Session

Feature	SparkContext	SparkSession
API Level	Low-level (RDD-based)	High-level (DataFrame/SQL + RDDs)
Introduced In	Spark 1.x	Spark 2.x onwards
Best Used For	RDD-based operations	Modern Spark apps with DataFrames and SQL
Accessed By	<code>SparkContext</code>	<code>spark.sparkContext</code> (through SparkSession)

How many executors are there in worker node in spark?



Need to verify this:- Databricks clusters primarily use Spark's Standalone cluster manager.

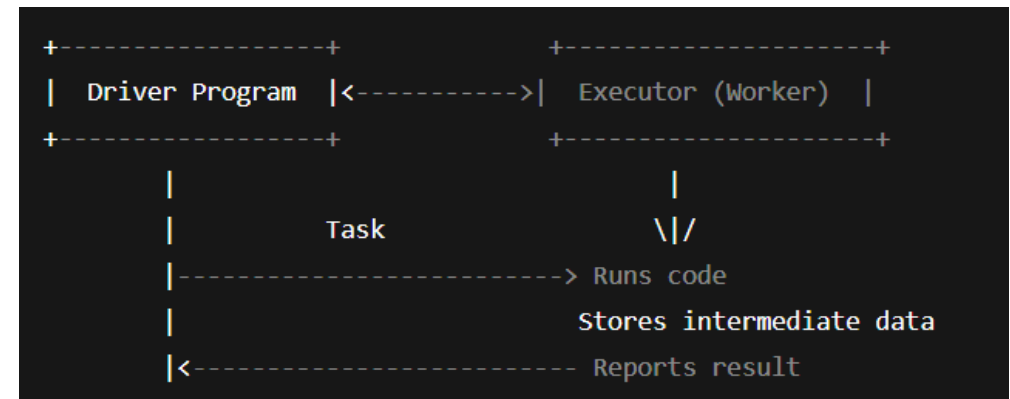
What Does an Executor Do?

Each executor:

- **Executes tasks** assigned by the driver (like map, reduce, filter).
- **Stores data** in memory or disk (for caching, shuffle, etc.).
- **Reports back** task status and results to the **driver**.

How Many Executors?

- Executors are **created per application**.
- You can have **multiple executors per worker node** (based on your configuration and resources).



Things to Remember:

- **Executors run on worker nodes.**
- You can have **multiple executors per worker, one executor per worker**, or **spread across nodes**, depending on your setup.
- Spark will **not oversubscribe** the resources. It tries to allocate executors based on available resources.

If you're using **YARN or Kubernetes**, the resource manager helps manage how many executors go to which nodes, and it may vary dynamically.

Cache In PySpark

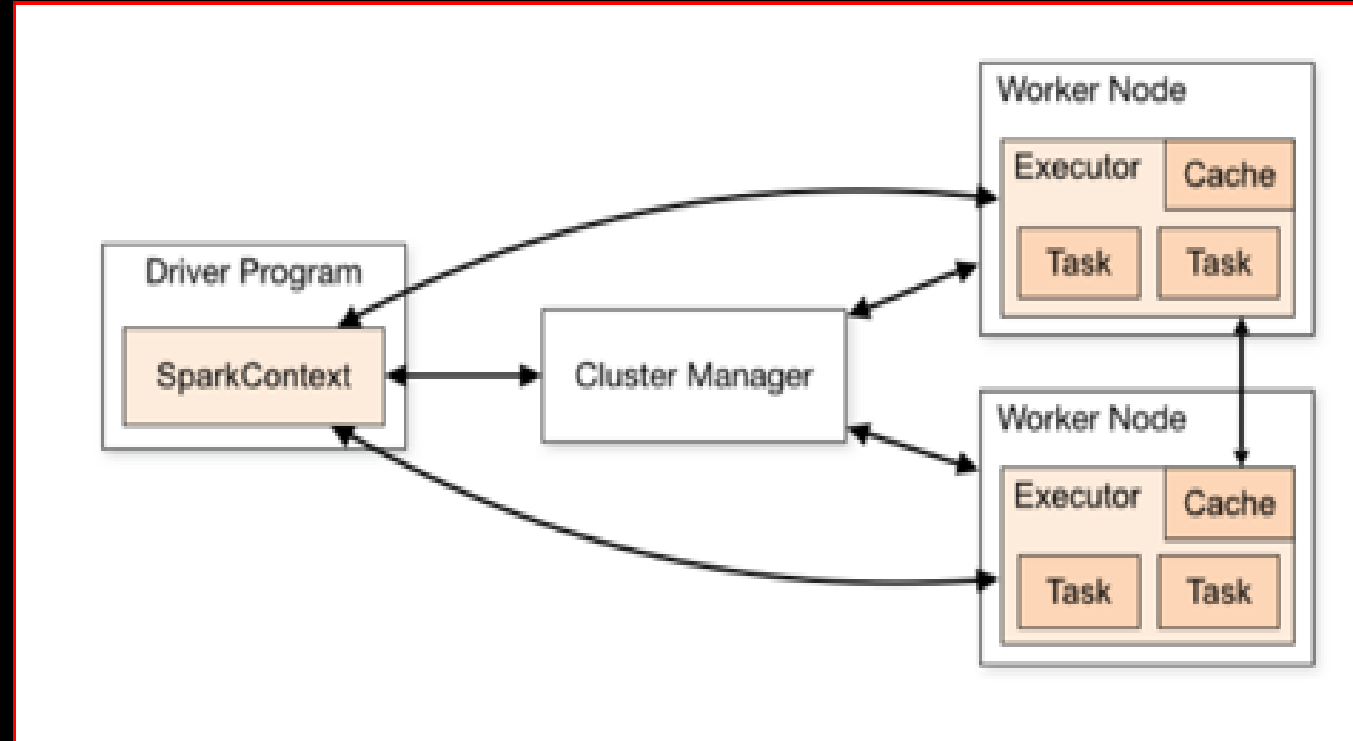
In **Apache Spark**, **cache in executors** refers to the **temporary in-memory storage** of RDDs or DataFrames **inside executor JVM memory**. This is done when you use `.cache()` or `.persist()` in your Spark code.

Spark Architecture – In Depth

- Spark Applications consist of:-
 1. a driver process
 2. a set of executor processes.

=>Master Slave Architecture

- Driver Node is the **Master**.
- Worker Nodes are **Slave**.



HADOOP vs SPARK

HADOOP ECOSYSTEM

Storage Unit | Compute Engine | Cluster Manager/Resource Manager

HDFS | **MAPREDUCE** | **YARN/MESOS/KUBERNETES/STANDALONE**



HDD(Hard Disk Drive)

SPARK ECOSYSTEM

HDFS | **SPARK** | **YARN/MESOS/KUBERNETES/STANDALONE**

ADLS Gen2

Amazon S3

Google Cloud Storage



SSD(Solid State Drive)



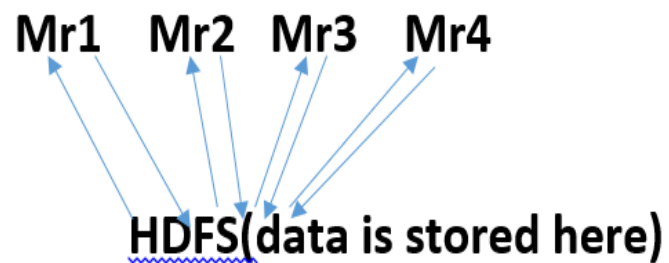
RAM(Random Access Memory)

In-memory computation->processing in RAM

General purpose->scala,python,spark sql,machine learning

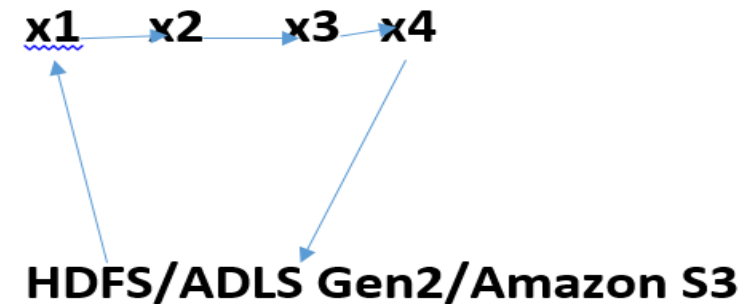
Note:-Spark is 10 to 100 faster than mapreduce as it uses in-memory computation(RAM) .

HADOOP



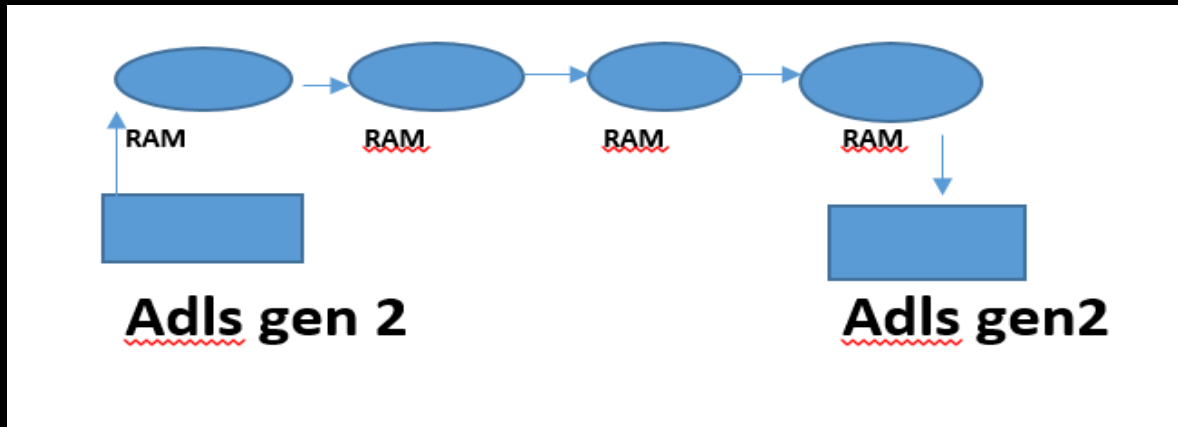
Multiple I/O operations

SPARK



Less I/O operations

- Input Output operations are very costly in big data.
- So,we always try to minimise it.
- Spark is 10 to 100 times faster than map reduce.
- Spark is not a language.It is a Compute Engine.



What is Hadoop?

- Apache Hadoop is an **open source framework** that is used to efficiently store and process large datasets ranging in size from gigabytes to petabytes of data.
- Instead of using one large computer to store and process the data, Hadoop allows clustering multiple computers to analyze massive datasets in parallel more quickly.

Main modules of Hadoop:

- **Hadoop Distributed File System (HDFS)** – A distributed file system that runs on standard or low-end hardware. HDFS provides better data throughput than traditional file systems, in addition to high fault tolerance and native support of large datasets.
- **Yet Another Resource Negotiator (YARN)** – Manages and monitors cluster nodes and resource usage.
- **MapReduce** – A framework that helps programs do the parallel computation on data. The map task takes input data and converts it into a dataset that can be computed in key value pairs. The output of the map task is consumed by reduce tasks to aggregate output and provide the desired result.

Some More Hadoop Ecosystem Components:

HBase:

NoSQL database built on HDFS.

Apache Sqoop:

Imports/exports **batch data** data between Hadoop and relational databases (MySQL, Oracle, etc.).

Apache Flume:

Apache Flume is a distributed, reliable, and available system used for efficiently collecting, aggregating, and moving large amounts of **streaming data** from various sources to a centralized data store like HDFS or HBase

Apache Hive:

SQL-like interface to query data in HDFS. Converts SQL queries into MapReduce/Spark jobs.

Apache Pig:

Scripting platform for analyzing large datasets. Uses **Pig Latin** language.

Apache Oozie:

Workflow scheduler for Hadoop jobs .

Spark Latest Version

 [Download](#) [Libraries](#) [Documentation](#) [Examples](#) [Community](#) [Developers](#) [GitHub](#) [Apache Software Foundation](#)

Download Apache Spark™

- Choose a Spark release:
- Choose a package type:
- Download Spark: [spark-4.0.0-bin-hadoop3.tgz](#)
- Verify this release using the 4.0.0 [signatures](#), [checksums](#) and [project release KEYS](#) by following these [procedures](#).

Note that Spark 4 is pre-built with Scala 2.13, and support for Scala 2.12 has been officially dropped. Spark 3 is pre-built with Scala 2.12 in general and Spark 3.2+ provides additional pre-built distribution with Scala 2.13.

Link with Spark

Spark artifacts are [hosted in Maven Central](#). You can add a Maven dependency with the following coordinates:

```
groupId: org.apache.spark  
artifactId: spark-core_2.13  
version: 4.0.0
```

Installing with PyPi

[PySpark](#) is now available in pypi. To install just run `pip install pyspark`.

Latest News

Spark 3.5.6 released (May 29, 2025)

Spark 4.0.0 released (May 23, 2025)

Spark 3.5.5 released (Feb 27, 2025)

Spark 3.5.4 released (Dec 20, 2024)

[Archive](#)


[DOWNLOAD SPARK](#)

Built-in Libraries:

- [SQL and DataFrames](#)
- [Spark Streaming](#)
- [MLlib \(machine learning\)](#)
- [GraphX \(graph\)](#)
- [Third-Party Projects](#)

DO I NEED TO LEARN HADOOP FOR DATA SCIENCE OR DATA ENGINEERING OR DATA ANALYST ROLE?





ForumDE
Code Your Career

New Azure batch starting on 26th July, 2025



Microsoft
Azure



Azure Data
Factory



databricks

PySpark

NEW BATCH

ANNOUNCEMENT

Updates of Our New

AZURE DATA ENGINEERING

COURSE Duration - 4 Months

Job Oriented IT Training



Mr. Arun Kumar



+91-70429 28331, +91-98018 30173



<https://forumde.in>

Official Link:- <https://spark.apache.org/docs/latest/rdd-programming-guide.html#actions>

show()

The `show()` method in PySpark is an action used to display the contents of a DataFrame in a tabular format.

reduce(*func*)

Aggregate the elements of the dataset using a function *func* (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.

collect()

Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.

count()

Return the number of elements in the dataset.

first()

Return the first element of the dataset (similar to `take(1)`).

take(*n*)

Return an array with the first *n* elements of the dataset.

takeSample(*withReplacement*, *num*, [*seed*])

Return an array with a random sample of *num* elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.

takeOrdered(*n*, [*ordering*])

Return the first *n* elements of the RDD using either their natural order or a custom comparator.

saveAsTextFile(*path*)

Write the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark will call `toString` on each element to convert it to a line of text in the file.

saveAsSequenceFile(*path*) (Java and Scala)

Write the elements of the dataset as a Hadoop SequenceFile in a given path in the local filesystem, HDFS or any other Hadoop-supported file system. This is available on RDDs of key-value pairs that implement Hadoop's Writable interface. In Scala, it is also available on types that are implicitly convertible to Writable (Spark includes conversions for basic types like Int, Double, String, etc).

saveAsObjectFile(*path*) (Java and Scala)

Write the elements of the dataset in a simple format using Java serialization, which can then be loaded using `SparkContext.objectFile()`.

countByKey()

Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.

foreach(*func*)

Run a function *func* on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems.

Note: modifying variables other than Accumulators outside of the `foreach()` may result in undefined behavior.

Official Link:- <https://spark.apache.org/docs/latest/rdd-programming-guide.html#actions>

map(*func*)

Return a new distributed dataset formed by passing each element of the source through a function *func*.

filter(*func*)

Return a new dataset formed by selecting those elements of the source on which *func* returns true.

flatMap(*func*)

Similar to map, but each input item can be mapped to 0 or more output items(so *func* should return a Seq rather than a single item).

mapPartitions(*func*)

Similar to map, but runs separately on each partition (block) of the RDD.

mapPartitionsWithIndex(*func*)

Similar to mapPartitions, but also provides *func* with an integer value representing the index of the partition.

sample(*withReplacement*, *fraction*, *seed*)

Sample a fraction *fraction* of the data, with or without replacement, using a given random number generator seed.

union(*otherDataset*)

Return a new dataset that contains the union of the elements in the source dataset and the argument.

intersection(*otherDataset*)

Return a new RDD that contains the intersection of elements in the source dataset and the argument.

distinct([*numPartitions*])

Return a new dataset that contains the distinct elements of the source dataset.

`groupByKey([numPartitions])`

When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.

Note: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using `reduceByKey` or `aggregateByKey` will yield much better performance.

Note: By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional **numPartitions** argument to set a different number of tasks.

`reduceByKey(func, [numPartitions])`

When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function `func`, which must be of type `(V,V) => V`.

aggregateByKey(*zeroValue*)(*seqOp*, *combOp*, [*numPartitions*])

When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a neutral "zero" value. Allows an aggregated value type that is different than the input value type, while avoiding unnecessary allocations. Like in `groupByKey`, the number of reduce tasks is configurable through an optional second argument.

sortByKey([*ascending*], [*numPartitions*])

When called on a dataset of (K, V) pairs where K implements `Ordered`, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean `ascending` argument.

join(*otherDataset*, [*numPartitions*])

When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through `leftOuterJoin`, `rightOuterJoin`, and `fullOuterJoin`.

cogroup(*otherDataset*, [*numPartitions*])

When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>))tuples. This operation is also called groupWith.

cartesian(*otherDataset*)

When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).

pipe(*command*, [*envVars*])

Pipe each partition of the RDD through a shell command, e.g. a Perl or bash script. RDD elements are written to the process's stdin and lines output to its stdout are returned as an RDD of strings.

coalesce(*numPartitions*)

Decrease the number of partitions in the RDD to *numPartitions*. Useful for running operations more efficiently after filtering down a large dataset.

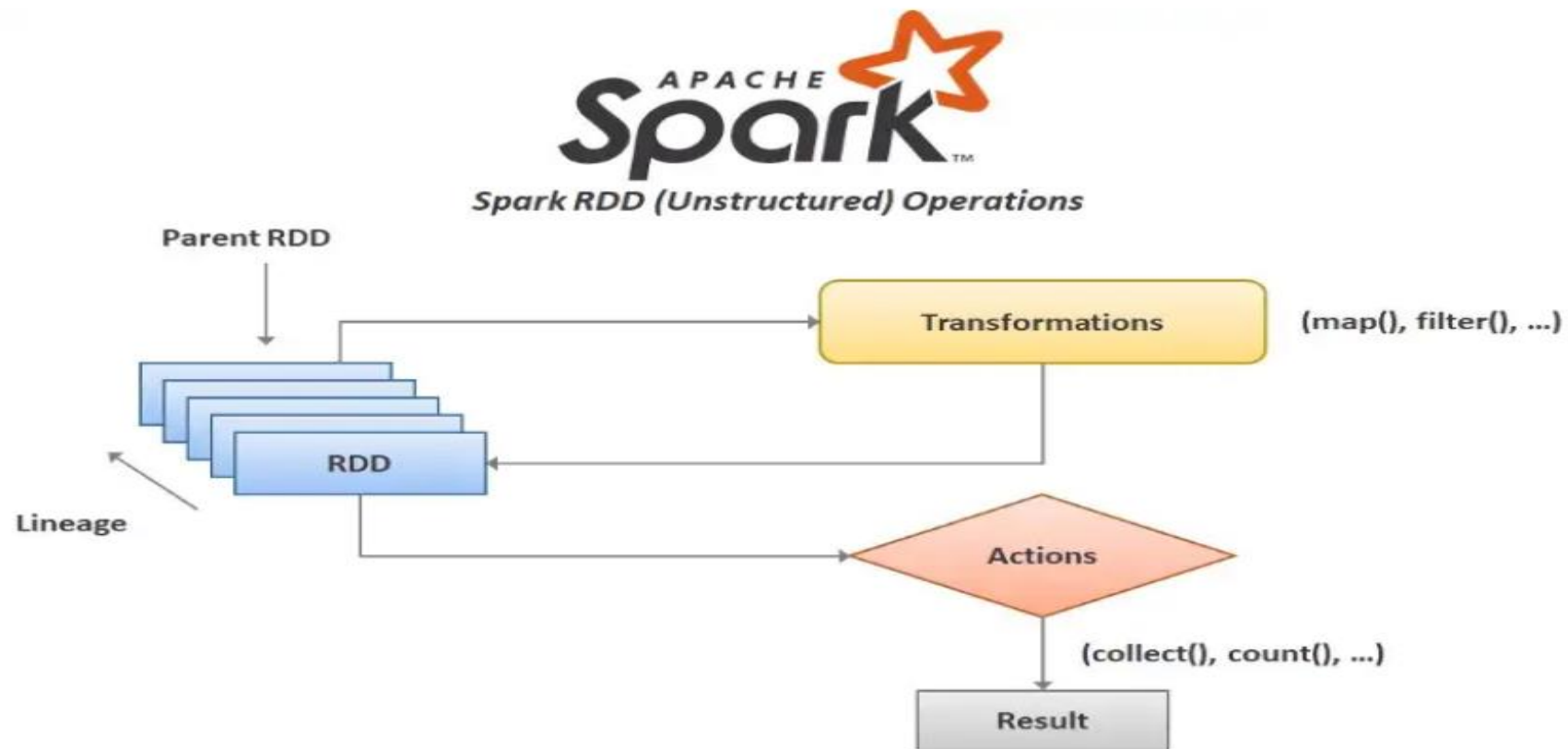
repartition(*numPartitions*)

Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.

repartitionAndSortWithinPartitions(*partitioner*)

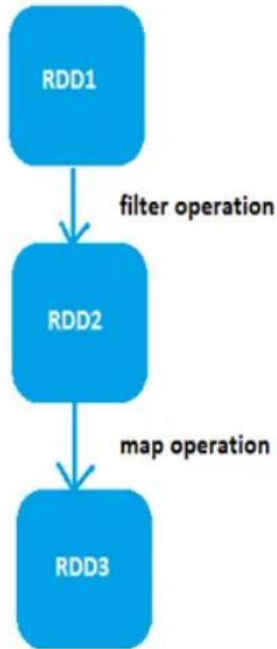
Repartition the RDD according to the given *partitioner* and, within each resulting partition, sort records by their keys. This is more efficient than calling *repartition* and then sorting within each partition because it can push the sorting down into the shuffle machinery.

RDD



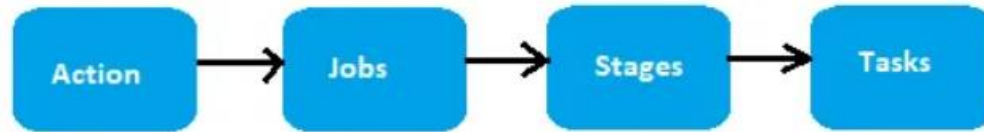
Spark RDD Operation Schema

- Lineage defines the transformation steps which are used to generate an RDD. Basically, it is a Logical Plan which tells us how to create an RDD by applying what transformation.
- The picture shows RDD2 is getting created by applying a filter operation on RDD1. Similarly when we use map operation on RDD2 then RDD3 will be generated.
- This information is very useful from dependency point of view and will help Spark to handle any failure, because it knows from the Lineage graph what all transformations are needed to generate the missing RDDs (during any failure).
- One more point to observe: Lineage **doesn't have any action defined**. It's a **pure transformation tracker**.
- Points to be noted:
 - a) Useful from dependency point of view
 - b) it's a Logical Plan
 - c) No Action



Job, Stages and Task

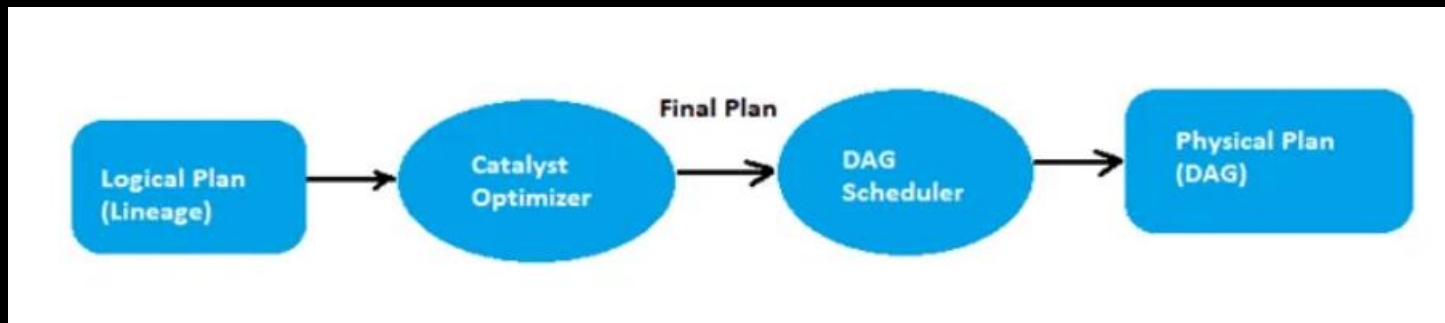
- Whenever spark encounters an Action it creates a JOB.
- Based on the transformation types STAGES are getting generated.
- Each STAGE contains TASKS to execute.



Level	Runs in Parallel?	Explanation
Stage	✗ No	Stages run sequentially . A stage cannot start until all its parent stages complete.
Task	✓ Yes	Tasks within a stage are the smallest unit of execution and run in parallel across executors.

DAG

- DAG (Directed Acyclic Graph) on the other hand is the **acyclic graph of Stages**.
- After applying multiple transformations on RDD when an Action is encountered, the Logical Plan (Lineage) is getting submitted into Catalyst Optimizer.



Logical Plan

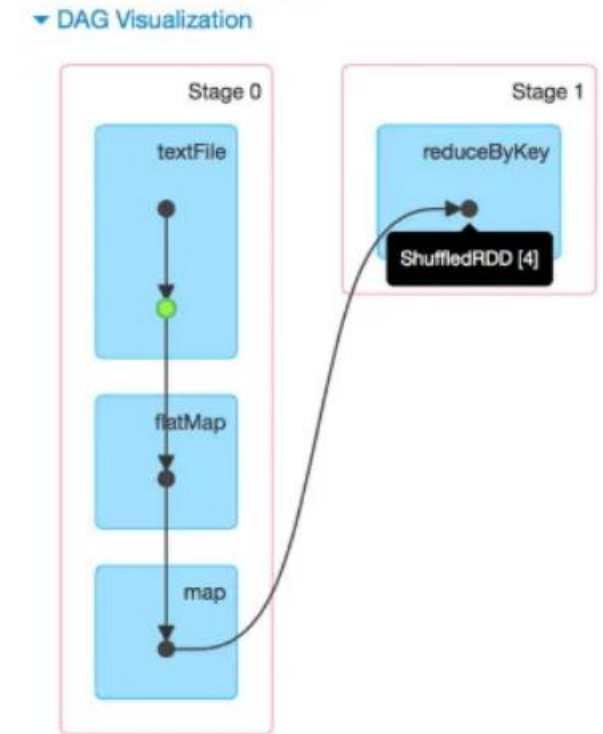
A high-level representation of your query. It represents what operations you want to perform.

Physical Plan

A low-level plan that tells Spark how to execute it. It represents how to perform those operations.

DAG Cont...

- Catalyst Optimizer will apply internal logic and submit the final plan to DAG Scheduler.
- **It's DAG Scheduler responsibility to generate Physical Plan (DAG) and splits the whole execution process into Stages.**
- In picture we can see one Spark Job has 2 different stages. Both the stages are having transformation details and also the dependency information between them.
- So, it's very clear that DAG is more informative from **Execution point of view**. It helps to identify **how different stages are dependent on each other**.
- Points to be noted:
 - a) Useful from Implementation and Execution point of view
 - b) it's a **Physical Plan**
 - c) **Always has an Action**



DAG Cont...

- The DAG is “directed” because the operations are executed in a specific order.
- “Acyclic” because there are no loops or cycles in the execution plan.

